

Ruby Chains 基本ルール

プレイ人数: 2~4人

プレイ時間目安: 30分 ~ 60分



1. ゲームの概要

1.1 ゲームの目的

自分以外の全てのプレイヤーのライフポイント (LP) を0にすること。

1.2 ゲームの勝利

他の全てのプレイヤーが敗北した時、残ったプレイヤーが**ゲームの勝利者**となります。

1.3 敗北条件

- プレイヤーのLPが0以下になった場合、即座に敗北します。
- 山札からカードを引く必要がある時に引けなかった場合、即座に敗北します。

2. ゲームの準備

2.1 ライフポイント (LP)

全プレイヤーは初期LPをセットします (推奨初期LP: 8~12)。LP は各プレイヤーで管理します。

2.2 共通山札の準備

全10種の「レシーバーカード」を裏向きでよくシャッフルし、共通の「レシーバー山札」として場の中央に置きます。

2.3 メソッドカードデッキ

後述の「3.2 デッキ構築ルール」に従った自分のデッキを裏向きでよくシャッフルし、自分の場に置きます。

2.4 手札

自分のデッキから手札を6枚引きます。

2.4 ゲーム開始

じゃんけんなどで最初のプレイヤーを決めます。

「レシーバーカード」を共通山札から1枚めくり、場の中央に置きます。

3. メソッドカードの種類とデッキ構築

3.1 ゲームの仕様

- 全てのメソッドおよびメソッドチェーンの挙動は、最新のRubyを `irb` で実行した結果を正とします。
- メソッドカードのQRコードは、公式ドキュメントへの補助として使用します。

※ 最終手段としての `irb` での確認よりも、まずはプレイヤー同士の話し合いで解決することを推奨します。

3.2 デッキ構築ルール

メソッドカードには以下の4カテゴリがあり、以下の上限枚数ルールに従って合計30枚のデッキを構築します。

主力	1種類あたり3枚以内
ユーティリティ	1種類あたり2枚以内
ジェネレーター	1種類あたり1枚以内
切り札	1種類あたり2枚以内



4. ゲームの基本フロー

4.1 自分のターン

自分のターン中、プレイヤーは以下のいずれかのアクションを1回行えます。アクション完了後、次のプレイヤーのターンに移ります。

- メソッドカードを場のメソッドチェーン最後尾のカードの右隣に繋げて1枚出す
- 「パス」を宣言する

※ 手札がない場合は強制的にパスをします。

4.2 メソッドカードのプレイ基本ルール

- 接続のルール

チェーン最後尾のカードの**実際の出力結果**（オブジェクト）に対して、手札から出すメソッドカードを実行時エラーなく呼び出せる場合にのみ、そのカードをプレイできます。

- 主力・ユーティリティ・ジェネレーターカード

現在のチェーンに対して引数なしで呼び出し（またはジェネレーターの固定引数で実行し）ても有効なものではありません。

- ジェネレーターカード

カードに記載された固定の引数で実行されます。

最新ルール、FAQ、カードリストなど

Github リポジトリ上で随時公開しています。

要チェック 📌



・ 切り札カード

単体で出す場合

挙動はRubyで `{ |it| it }` ブロックを渡した時と同じものとして扱います。

Proc引数と共に2枚で出す場合

切り札カード（例: `.map`）と、手札からもう1枚のメソッドカード（例: `.to_s`）を同時に場に出します（合計2枚消費）。

挙動は `.map(&.to_s)` と同じです。Proc引数として指定するメソッドは、Rubyの `&` 記法として有効なものでなければなりません。

4.3 プレイの無効化（エラー回避）

プレイヤーがメソッドカードを出した時、他のプレイヤーは4.2の「接続のルール」に違反する（＝実行時エラーを引き起こす）と判断した場合、エラーの可能性を指摘できます。

指摘されたプレイヤーと指摘したプレイヤーでエラーについて合意するか、`irb` で確認してエラーが発生する場合、出そうとしたカードを手札に戻します。

指摘されたプレイヤーは、改めてメソッドカードを出すか、パスを宣言します。

5. ラウンドの終了

誰か1人を除き、他のプレイヤー全員が「パス」を宣言したら、ラウンド終了となります。

最後にカードを出したプレイヤーを、そのラウンドの「ラウンド勝利者」とします。

※ 全プレイヤーがカードを出せずにパスした場合、そのラウンドは「勝利者なし」で終了します。

6. ラウンドの終了処理

ラウンドが終了したら、以下の処理を順番に行います。

6.1 ダメージ計算

ラウンド勝利者がいる場合、ラウンド勝利者以外の全てのプレイヤーはLPに2ダメージを受けます。LPが0以下になったプレイヤーは、このラウンドで敗北します。

※「勝利者なし」の場合、このステップではダメージは発生しません。

6.2 捨て札処理

場のメソッドチェーン、レシーバーカードを全て捨て札にします。

プレイヤーが出したカードは自分の捨て札置き場に置きます。

6.3 ドロー

各プレイヤーは、自分のデッキからカードを2枚引きます。

6.4 次のラウンドへ

- ・ 新しいレシーバーカードをめくり、次のラウンドを開始します。「レシーバー山札」がなくなった場合は、捨て札をシャッフルして共通山札に戻します。
- ・ 前のラウンドのラウンド勝利者から逆順で開始します。「勝利者なし」だった場合は、前のラウンドで最初にプレイしたプレイヤーの、次のプレイヤーから開始します。

プレイ例

- ① レシーバーカード “1, 2, 3, 4, 5”
- ② プレイヤーA のカード `.split`
⇒ `["1,2,3,4,5"]`
- ③ プレイヤーB のカード `.map(&.to_i)`
⇒ `[1,2,3,4,5]`
- ④ プレイヤーC のカード `.join`
⇒ “12345”
他の全員がパスするまでチェーンは続く

※ 上級ルールの場合

- ⑤ 要求カード

上: 全て大文字の String にせよ。

下: 全ての要素が Integer である Array にせよ。

上下どちらかのお題を達成した状態で勝利すれば、LP4ダメージ



慣れてきたら上級ルールでも遊んでみよう！

基本ルールに以下の4つの大きな変更点を加え、より戦略的で緊張感のある「お題達成」「コードレビュー」と「リソース管理」の駆け引きを楽しめるようになります。

- ・ LPダメージルールの変更: 要求カード
- ・ 新リソース「KP (ナレッジポイント)」の導入
- ・ エラー判定の変更: 「コードレビュー」へ
- ・ 勝利への新ルート: 「バーストチェーン」

